

Below is a concise list of key takeaways from the lecture that you should know for the exam:

Understanding the Function Properties

- **Proper, l.s.c., and Coercive:**
 - *Proper*: The function has at least one point where its value is finite.
 - *Lower Semi-Continuous (l.s.c.)*: The function does not suddenly drop below nearby values, which helps in ensuring minimizers are preserved.
 - *Coercive*: The function grows to $+\infty$ as $\|x\| \rightarrow \infty$, preventing minimizers from "escaping" to infinity and ensuring existence.
 - **Convexity vs. Strict Convexity:**
 - *Convexity*: Every local minimizer is a global minimizer.
 - *Strict Convexity*: Guarantees a unique minimizer.
-

Differentiability Concepts

- **Gateaux Differentiability:**
 - Provides a directional derivative in every direction, leading to the definition of the gradient $\nabla f(x)$ used in first-order methods.
 - **Optimality Conditions:**
 - *Fermat's Rule*: For a differentiable function, if x^* is a minimizer, then $\nabla f(x^*) = 0$.
-

Smoothness and Convergence

- **L-smoothness:**
 - Ensures the gradient is Lipschitz continuous ($\|\nabla f(w) - \nabla f(z)\| \leq L\|w - z\|$).
 - This property is critical for selecting a proper step size in gradient descent and guaranteeing convergence.
- **Strong Convexity:**
 - A stronger condition than convexity that leads to faster (often linear or exponential) convergence rates.
- **Gradient Descent (GD):**
 - Understand the update rule: $x_{k+1} = x_k - \tau \nabla f(x_k)$
 - The step size τ must satisfy $0 < \tau < \frac{2}{L}$ to ensure descent.

- **Accelerated Methods (AGD):**
 - Incorporate momentum (e.g., Nesterov's method) to achieve an improved convergence rate of $O(1/k^2)$ compared to the $O(1/k)$ rate for standard GD.

Regularisation in Inverse Problems

- **Maximum Likelihood:**
 - For a Gaussian noise model, solve: $\min_x \frac{1}{2} \|Ax - y\|^2$.
- **Tikhonov Regularisation:**
 - Stabilizes ill-posed problems by adding a penalty term: $\min_x \left\{ \frac{1}{2} \|Ax - y\|^2 + \lambda \|Bx\|^2 \right\}$.
 - *Ridge Regularisation:* When B is the identity, penalizes large values of x .
 - *Sobolev Regularisation:* When B is a derivative operator, penalizes high gradients to promote smoothness.
- **Parameter Balancing:**
 - The regularisation parameter λ controls the trade-off between fitting the data and enforcing the prior (smoothness, sparsity, etc.).

Algorithmic Details

- **GD and AGD Update Rules:**
 - Be able to write and explain the update rules for both GD and AGD.
 - Understand how the step size is chosen based on the Lipschitz constant L .
 - Know the convergence rates and how they depend on the properties of f (e.g., convexity, strong convexity, and smoothness).

Keep this summary handy and review these points to solidify your understanding. Good luck with your exam!

Here is a **detailed summary and explanation** of the **Forward-Backward Splitting (FBS) algorithm** and its **accelerated variant (FISTA)**, covering all key concepts, mathematical formulations, and their importance for an exam.

1. Introduction to Forward-Backward Splitting (FBS)

Why do we need FBS?

- We often encounter optimization problems where the objective function is **composed of two terms**:

$$\min_{x \in \mathbb{R}^n} F(x) = f(x) + g(x)$$

where:

- $f(x)$ is **smooth, convex, and differentiable**, e.g., least squares loss.
 - $g(x)$ is **non-smooth, convex, but typically "proximable"**, e.g., ℓ_1 -norm for sparsity.
- Why not just use gradient descent?**
 - Gradient descent **requires differentiability**, but $g(x)$ is often **non-differentiable** (e.g., ℓ_1 -norm or indicator functions of constraints).
 - Proximal methods allow handling non-smooth terms **efficiently**.

Solution Approach: Combining Two Ideas

- Gradient Descent for $f(x)$** : Since $f(x)$ is smooth, we can use its **gradient**.
- Proximal Operator for $g(x)$** : Since $g(x)$ is non-smooth, we use its **proximal operator**.

This leads to the **Forward-Backward Splitting (FBS) algorithm**, also called **Proximal Gradient Descent**.

2. Forward-Backward Splitting Algorithm

Mathematical Formulation

The **FBS update rule** is given by:

$$x_{k+1} = \text{prox}_{\tau g}(x_k - \tau \nabla f(x_k))$$

where:

- τ is the **step size** and should satisfy $0 < \tau < \frac{2}{L}$, where L is the **Lipschitz constant** of $\nabla f(x)$.
- The **proximal operator** is defined as:

$$\text{prox}_{\tau g}(x) = \arg \min_{y \in \mathbb{R}^n} (g(y) + \frac{1}{2\tau} \|y - x\|^2)$$
which **smoothly handles the non-smooth term $g(x)$** .

Interpretation

1. **Forward Step:** $x_{k+1} = x_k - \tau \nabla f(x_k)$
 - This is a **gradient step** using $\nabla f(x)$, like standard gradient descent.
2. **Backward Step:** $x_{k+1} = \text{prox}_{\tau g}(x_k - \tau \nabla f(x_k))$
 - This enforces the structure imposed by $g(x)$ (e.g., sparsity if $g(x) = \|x\|_1$).
 - Acts like a **"soft constraint"**.

Why is it called "Forward-Backward"?

- The **forward step** is a **gradient descent step** for $f(x)$.
 - The **backward step** applies the **proximal operator** of $g(x)$.
-

3. Convergence Properties

- **FBS has a convergence rate of:**

$$F(x_k) - F(x^*) = O(1/k)$$

- Meaning the error decreases at a **rate of $1/k$** , which is the same as standard gradient descent.
 - It **converges, but slowly**.
- **Issues with Basic FBS**
 - It requires many iterations for convergence.
 - We need **momentum** to accelerate convergence.
-

4. Accelerated Forward-Backward Splitting (FISTA)

To speed up FBS, we use **Nesterov acceleration**.

FISTA Algorithm (Fast Iterative Shrinkage-Thresholding Algorithm)

Instead of directly updating x_k , we introduce **momentum**:

1. **Compute an extrapolated point:** $y_k = x_k + t_k(x_k - x_{k-1})$
 - Adds a momentum term to accelerate convergence.
2. **Perform the proximal update:** $x_{k+1} = \text{prox}_{\tau g}(y_k - \tau \nabla f(y_k))$
 - Same as FBS, but applied to y_k instead of x_k .
3. **Update the momentum parameter:** $t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$

FISTA Convergence Rate

$$F(x_k) - F(x^*) = O(1/k^2) \quad F(x_k) - F(x^*) = O(1/k^2)$$

- **FISTA is much faster** than standard FBS.
 - **Key observation:** The $O(1/k^2)$ rate is the best possible for first-order convex optimization.
-

5. Special Cases and Applications

1. If $g(x) = 0$: Gradient Descent

$$x_{k+1} = x_k - \tau \nabla f(x_k) \quad x_{k+1} = x_k - \tau \nabla f(x_k)$$

- FBS reduces to **Gradient Descent (GD)**.

2. If $g(x)$ is an Indicator Function $\iota_C(x)$

- Proximal step becomes **Projection**: $x_{k+1} = P_C(x_k - \tau \nabla f(x_k))$
- This is called **Projected Gradient Descent (PGD)**.

3. If $g(x) = \lambda \|x\|_1$: ISTA (Iterative Soft-Thresholding Algorithm)

$$x_{k+1} = S_{\lambda\tau}(x_k - \tau \nabla f(x_k)) \quad x_{k+1} = S_{\lambda\tau}(x_k - \tau \nabla f(x_k))$$

where $S_{\lambda\tau}(\cdot)$ is the **soft-thresholding operator**:

$$S_{\lambda\tau}(z) = \text{sign}(z) \max(|z| - \lambda\tau, 0) \quad S_{\lambda\tau}(z) = \text{sign}(z) \max(|z| - \lambda\tau, 0)$$

- **ISTA is widely used in compressed sensing and LASSO regression.**
 - **FISTA improves ISTA's convergence from $O(1/k)$ to $O(1/k^2)$.**
-

6. Exam Preparation: Important Concepts

Key Definitions

1. **Proximal Operator:** $\text{prox}_{\tau g}(x) = \arg \min_y (g(y) + \frac{1}{2\tau} \|y - x\|^2)$
2. **Forward-Backward Splitting (FBS):**
 - Combines gradient descent and proximal steps.
 - Converges at $O(1/k)$.
3. **FISTA (Accelerated FBS):**
 - Adds momentum (Nesterov acceleration).
 - Converges at $O(1/k^2)$.

Common Exam Questions

1. Derive the FBS update rule and explain its components.
2. What is the proximal operator? Provide an example.
3. Prove that FBS converges at $O(1/k)O(1/k)$.
4. Compare FBS and FISTA in terms of convergence speed.
5. Explain why ISTA is useful for LASSO regression.

7. Summary Table

Method	Update Rule	Convergence Rate
Gradient Descent	$x_{k+1} = x_k - \tau \nabla f(x_k)$	$O(1/k)O(1/k)$
Projected GD	$x_{k+1} = PC(x_k - \tau \nabla f(x_k))$	$O(1/k)O(1/k)$
FBS	$x_{k+1} = \text{prox}_{\tau g}(x_k - \tau \nabla f(x_k))$	$O(1/k)O(1/k)$
ISTA	$x_{k+1} = S_{\lambda \tau}(x_k - \tau \nabla f(x_k))$	$O(1/k)O(1/k)$
FISTA	Uses momentum + prox step	$O(1/k^2)O(1/k^2)$

This **detailed summary** should prepare you well for any exam question on FBS, FISTA, and proximal optimization. Let me know if you need further clarifications! 🚀

5. Acceleration Strategies (Continued)

Momentum Methods

- **Heavy Ball Acceleration (Deterministic Case):** $x_{k+1} = x_k - \tau \nabla f(x_k) + \beta_k (x_k - x_{k-1})$, $x_{k+1} = x_k - \tau \nabla f(x_k) + \beta_k (x_k - x_{k-1})$, where $\beta_k \in (0, 1)$ is the momentum parameter.
- **Stochastic Momentum Method:**
In the stochastic setting, momentum is applied as:
 $x_{k+1} = x_k - \tau \nabla f_{i_k}(x_k) + \beta_k (x_k - x_{k-1})$, where i_k is randomly chosen at each step.
- **Theorem (Convergence of SGD with Momentum):**
By choosing: $\tau_k = \frac{2\eta}{k+3}$, $\beta_k = \frac{k}{k+1}$, with $\eta \leq \frac{1}{4L_{\max}}$, $\tau_k = \frac{2\eta}{k+3}$, $\beta_k = \frac{k}{k+1}$, the following bound holds: $\mathbb{E}[f(x_k) - f(x^*)] \leq \frac{\|x_0 - x^*\|^2}{2\eta(k+1)} + 2\eta \text{Var}[\nabla f_i(x^*)]$.
 - This means that the function values $f(x_k)$ converge, but not necessarily the iterates themselves.
 - **Key takeaway:** Momentum speeds up convergence in practice but requires careful tuning of τ_k and β_k .

6. Adaptive Step-Sizes and Non-Smooth Optimization

Variants of SGD

- **Averaging:**
 - SGD with averaging (Polyak-Ruppert averaging) is closely related to SGD with momentum.
- **AdaGrad & RMSProp:**
 - **AdaGrad:** Adjusts learning rates per parameter based on past gradient magnitudes.
 - **RMSProp:** Normalizes learning rates using a moving average of squared gradients.
- **Adam (Adaptive Moment Estimation):**
 - Uses both first and second moment estimates of the gradient to adjust step-sizes adaptively.

7. Stochastic Proximal Gradient Descent (SPGD)

Used for problems with non-smooth terms:

$$\min_{x \in \mathbb{R}^d} (1/n \sum_{i=1}^n f_i(x) + g(x)), \quad \min_{x \in \mathbb{R}^d} \left(\frac{1}{n} \sum_{i=1}^n f_i(x) + g(x) \right),$$

where:

- $f_{i,j}$ is differentiable with an L -Lipschitz gradient.
- g is a convex but possibly non-differentiable function.

SPGD Update Rule

Instead of a simple gradient step, apply a proximal update:

$$x_{k+1} = \text{prox}_{\tau g}(x_k - \tau \nabla f_{i,j}(x_k)), x_{k+1} = \operatorname{prox}_{\tau g}(x_k - \tau \nabla f_{i,j}(x_k)),$$

where the **proximal operator** is defined as:

$$\text{prox}_{\tau g}(z) = \arg \min_x \{g(x) + \frac{1}{2\tau} \|x - z\|^2\}. \operatorname{prox}_{\tau g}(z) = \arg \min_x \left\{ g(x) + \frac{1}{2\tau} \|x - z\|^2 \right\}.$$

Convergence of SPGD

- If $\tau < \frac{1}{4L_{\max}}$, the expected function value satisfies:
$$E[F(x_k) - F(x^*)] \leq \|x_0 - x^*\|^2 + 2\tau(F(x_0) - F(x^*))^2(1 - 4\tau L_{\max})^k + 2\tau \text{Var}[\nabla f_{i,j}(x^*)](1 - 4\tau L_{\max})^k + \frac{1}{2\tau} \text{Var}[\nabla f_{i,j}(x^*)](1 - 4\tau L_{\max})^k$$
 - This shows that, on average, the method converges at a rate of $O(1/k)$, but careful tuning of τ is needed.
 - Acceleration via momentum in the stochastic proximal setting is challenging.

8. Summary of Convergence Comparisons

Method	Pros	Cons
SGD (Constant Step-Size)	Simpler implementation, $O(1/k)$ rate	Variance limits final accuracy
SGD with Averaging	Reduces oscillations, better function value convergence	Still $O(1/k)$
Momentum SGD	Can accelerate convergence	Needs careful tuning, may overshoot
Adaptive SGD (Adam, RMSProp)	More robust for noisy or sparse data	More parameters to tune
SPGD (Proximal Gradient)	Works for non-smooth problems	Acceleration strategies unclear

9. Key Takeaways for Your Exam

1. **L-Smoothness** ensures bounded gradient changes, crucial for proving convergence.

2. **SGD** reduces computational cost by using stochastic gradients, but it introduces variance.
3. **SGD Averaging** (Polyak-Ruppert) helps stabilize iterates and improve function value convergence.
4. **Momentum Methods** improve convergence but require careful tuning.
5. **Adaptive SGD (e.g., Adam, RMSProp)** dynamically adjusts step-sizes, useful for complex datasets.
6. **Proximal SGD** extends stochastic optimization to non-smooth problems.
7. **Convergence Trade-offs:**
 - Faster methods (momentum) may increase oscillations.
 - Averaging and adaptive methods help but require extra computation.

This summary should prepare you well for the exam! 🚀 Let me know if you need any specific explanations.

Here's a **detailed summary** of the lecture "**Bridging Optimization & Learning: Unrolling and Plug & Play Approaches**" by Luca Calatroni. Since you're preparing for an exam, I'll explain the concepts clearly and highlight their significance.

Detailed Summary of Lecture 7: Bridging Optimization & Learning

Instructor: Luca Calatroni

Course: MSc DSAI - Université Côte d'Azur

Topic: Unrolling and Plug & Play (PnP) Approaches in Optimization & Deep Learning

Application Area: Inverse Problems in Biological Imaging

1. Unrolled Algorithms

Unrolled algorithms refer to iterative optimization techniques that can be **interpreted as deep neural networks** when unfolded over multiple iterations. This connection enables us to **learn optimization parameters** instead of fixing them manually.

1.1 Back to PGD, FB, and ISTA

The lecture begins by revisiting basic optimization methods:

- **Proximal Gradient Descent (PGD)** and **Forward-Backward Splitting (FB)** are iterative algorithms used for solving regularized optimization problems.
- **Iterative Soft Thresholding Algorithm (ISTA)** is a specific case of PGD used for sparse recovery.

Optimization Problem Formulation

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - y\|^2 + \lambda \|x\|_1$$

- **First term:** Data fidelity (measuring how well x fits the observation y).
- **Second term:** ℓ_1 -regularization (sparsity constraint).

ISTA Iterations

ISTA updates x_k iteratively:

$$x_{k+1} = \text{prox}_{\tau\lambda\|\cdot\|_1}(x_k - \tau A^T(Ax_k - y))$$

where:

- **Soft-thresholding function $T_{\tau\lambda\|\cdot\|_1}$:**

$$T_{\tau\lambda\|\cdot\|_1}(z_j) = \text{sign}(z_j) \max(|z_j| - \tau\lambda, 0)$$

📌 Key Questions:

- What happens if we stop at a fixed K iterations instead of running until convergence?
→ This leads to **algorithmic unrolling**.

2. Neural Network Interpretation of ISTA

ISTA can be **viewed as a recurrent neural network (RNN)**:

$$x_{k+1} = T_{\tau\lambda}(x_k - \tau A^T(Ax_k - y)) \quad x_{k+1} = T_{\{\tau\lambda\}}(x_k - \tau A^T(Ax_k - y))$$

which resembles:

$$x_{k+1} = \text{ReLU}((I - \tau A^T A)x_k + \tau A^T y) \quad x_{k+1} = \text{ReLU}((I - \tau A^T A)x_k + \tau A^T y)$$

- The **linear term** resembles a **fully connected layer**.
- The **thresholding function** acts like an **activation function** (e.g., ReLU).
- Each iteration **resembles a layer of a neural network**.

Key Insight:

If we **unroll ISTA for KK steps**, we obtain a **K-layer network** where the same parameters $A, \tau, \lambda, A, \tau, \lambda$ are **shared across all layers**, resembling **recurrent neural networks (RNNs)**.

3. Learned ISTA (LISTA)

Instead of using fixed parameters, LISTA **learns** the transformation matrices W_t, W_e, W_t, W_e and step size τ through training.

$$L(W_t, W_e, \tau) = \frac{1}{2S} \sum_{i=1}^S \|x_{K-1}^{(i)}(W_t, W_e, \tau; y_i) - x_i\|^2 \quad L(W_t, W_e, \tau) = \frac{1}{2S} \sum_{i=1}^S \|x_{K-1}^{(i)}(W_t, W_e, \tau; y_i) - x_i\|^2$$

where:

- $x_{K-1}^{(i)}$ is the predicted solution after KK iterations.
- Training is done using **stochastic gradient descent (SGD)**.

Advantages of LISTA

- Requires **fewer iterations** than ISTA to achieve a good solution.
 - **Faster convergence** due to learned parameters.
-

4. Image Denoising and Proximal Operators

4.1 The Image Denoising Problem

Given a noisy image y , the goal is to recover a **noise-free** image x :

$$y = x + b, b \sim \mathcal{N}(0, \sigma^2 I) \quad y = x + b, b \sim \mathcal{N}(0, \sigma^2 I)$$

where b is **Gaussian noise**.

4.2 Optimization Formulation

$$\min_x \frac{1}{2} \|x - y\|^2 + g(x) \quad \min_x \frac{1}{2} \|x - y\|^2 + g(x)$$

where:

- $g(x)g(x)$ is a **regularization term** enforcing prior knowledge (e.g., sparsity, smoothness).
- The **proximal operator** is used to solve this efficiently:

$$x^* = \text{prox}_{\sigma^2 g}(y)$$

which means x^* is obtained by applying a **denoising function** to y .

5. Plug & Play (PnP) Approaches

5.1 Key Idea

Instead of explicitly defining $g(x)g(x)$, we replace the **proximal operator** with a **pre-trained denoiser $D(y)D(y)$** :

$$x_{k+1} = D(x_k - \tau A^T(Ax_k - y)) \quad x_{k+1} = D(x_k - \tau A^T(Ax_k - y))$$

where D can be **any denoising function**, including:

- Traditional denoisers (e.g., BM3D, wavelets)
- **Deep learning-based denoisers (CNNs)**

5.2 Why is PnP Useful?

- We **don't need to define $g(x)g(x)$** explicitly.
- **Any off-the-shelf denoiser** can be used without modifying the optimization framework.

6. Regularization by Denoising (RED)

A more structured way of incorporating denoising into optimization is **Regularization by Denoising (RED)**.

$$x_{k+1} = x_k - \tau(A^T(Ax_k - y) + \lambda(x_k - D(x_k))) \quad x_{k+1} = x_k - \tau(A^T(Ax_k - y) + \lambda(x_k - D(x_k)))$$

- The term $x_k - D(x_k)x_k - D(x_k)$ enforces **denoising as a regularization**.
- Works well when $D(x)D(x)$ has a **symmetric Jacobian**.

📌 RED vs. PnP:

- **PnP**: Uses an arbitrary denoiser $D(x)D(x)$ without an explicit optimization function.
- **RED**: Defines an explicit function $g(x)g(x)$ using the denoiser.

7. Unrolling + Plug & Play

7.1 Combining LISTA and PnP

$$x_{k+1} = D_\theta((I - \tau A^T A)x_k + \tau A^T y) \quad x_{k+1} = D_\theta((I - \tau A^T A)x_k + \tau A^T y)$$

where $D_{\theta D_{\theta}}$ is a **learnable CNN denoiser**.

Advantages

- More **flexible and adaptive** than LISTA.
- **Faster convergence** than traditional PnP.
- No need to **manually tune regularization parameters**.

Challenges:

- Which parameters should we learn?
 - How many layers K should we use?
 - Does the learned approach still solve an optimization problem?
-

8. Conclusion & Key Takeaways

- ISTA can be unrolled into a deep network.
- LISTA learns **optimal parameters**, making convergence faster.
- Plug & Play (PnP) methods replace regularization with learned denoisers.
- RED provides a more structured way to use denoisers in optimization.
- Unrolling + PnP combines deep learning and optimization efficiently.

This lecture **bridges optimization and deep learning** by showing how iterative algorithms can be **reinterpreted as neural networks** and **improved using learned models**